



UNIVERSIDADE DO VALE DO ITAJAÍ
CIÊNCIA DA COMPUTAÇÃO – PROJETO DE SISTEMAS DIGITAIS
PROF. PhD. DOUGLAS ROSSI DE MELO

GUSTAVO HENRIQUE STAHL MÜLLER

PROJETO RTL

ITAJAÍ
2022

SUMÁRIO

1	INTRODUÇÃO.....	3
2	DESENVOLVIMENTO.....	4
2.1	ALGORITMO MDC.....	4
2.2	PROJETO.....	5
2.2.1	ETAPA 1 - CAPTURAR O COMPORTAMENTO EM CÓDIGO C	5
2.2.2	ETAPA 2 - CONVERTER PARA MÁQUINA DE ESTADOS.....	5
2.2.3	ETAPA 3 - CRIAR O CAMINHO DE DADOS	6
2.2.3.1	COMPARADOR DE 1 BIT	6
2.2.3.2	COMPARADOR DE 8 BIT	8
2.2.3.3	CAMINHO DE DADOS	11
2.2.4.	ETAPA 4 - DERIVAR A FSM DO CONTROLADOR.....	14
2.2.5	ETAPA 5 - CONEXÃO DO CONTROLADOR E CAMINHO DE DADOS.....	19
2.3	ANÁLISE E TESTES.....	22
2.3.1	PRIMEIRO TESTE	24
2.3.2	SEGUNDO TESTE.....	24
2.3.3	TERCEIRO TESTE	25

1 INTRODUÇÃO

Esse trabalho tem como objetivo consolidar os conhecimentos de Projeto RTL com o uso de VHDL, Quartus Prime e Questa, abrangendo a transformação de um algoritmo em C de cálculo de Máximo Divisor Comum em um processador correspondente, que possui um controlador e um caminho de dados.

A transformação foi realizada de acordo com as etapas de Projeto RTL descritas no livro “Digital Design with RTL Design, VHDL, and Verilog”, de Frank Vahid.

2 DESENVOLVIMENTO

2.1 ALGORITMO MDC

O algoritmo MDC (Máximo Divisor Comum) encontra o máximo divisor comum entre dois números de entrada. O cálculo iterativamente subtrai o menor número do maior número, até que eles sejam iguais.

O processador será composto de duas entradas de dados de 8 bits, que representam os números (*i_X* e *i_Y*), uma entrada para começar o processamento (*i_GO*), uma saída de dados de 8 bits (*o_D*) e uma saída para indicar quando o processador está pronto para começar (*o_RDY*).

Quando o algoritmo está esperando a entrada do usuário para começar (*i_GO*), o sinal de pronto (*o_RDY*) é igual à 1. Durante o processamento, o sinal *o_RDY* é igual à 0. Após o processamento, o algoritmo deve esperar novamente o sinal *i_GO* para que processe duas novas entradas.

Figura 1 – Código em C do algoritmo MDC.

```

1  int x, y; // variáveis internas
2
3  while (1) {
4      o_rdy = 1; // o_rdy: saída de 1 bit
5
6      while (!i_go); // i_go: entrada de 1 bit
7
8      x = i_x; // i_x : entrada de 8 bits
9      y = i_y; // i_y : entrada de 8 bits
10
11     o_rdy = 0;
12
13     while (x != y) {
14         if (x < y)
15             y = y - x;
16         else
17             x = x - y;
18     }
19
20     o_d = x; // o_d: saída de 8 bits
21 }
```

Fonte: O autor.

2.2 PROJETO

Para transformar o algoritmo em C em um processador correspondente, serão seguidas as 5 etapas de Projeto RTL do livro “Digital Design with RTL Design, VHDL, and Verilog”, de Frank Vahid.

2.2.1 ETAPA 1 - CAPTURAR O COMPORTAMENTO EM CÓDIGO C

O algoritmo em C que descreve o comportamento do processador já foi definido, conforme demonstrado no capítulo 2.1.

2.2.2 ETAPA 2 - CONVERTER PARA MÁQUINA DE ESTADOS

Esta etapa consiste em traduzir o código de alto-nível em uma máquina de estados de alto nível (HLSM). Para fazer isso, é necessário iterar sobre cada sentença do código em C e criar os estados e transições correspondentes na máquina de estados.

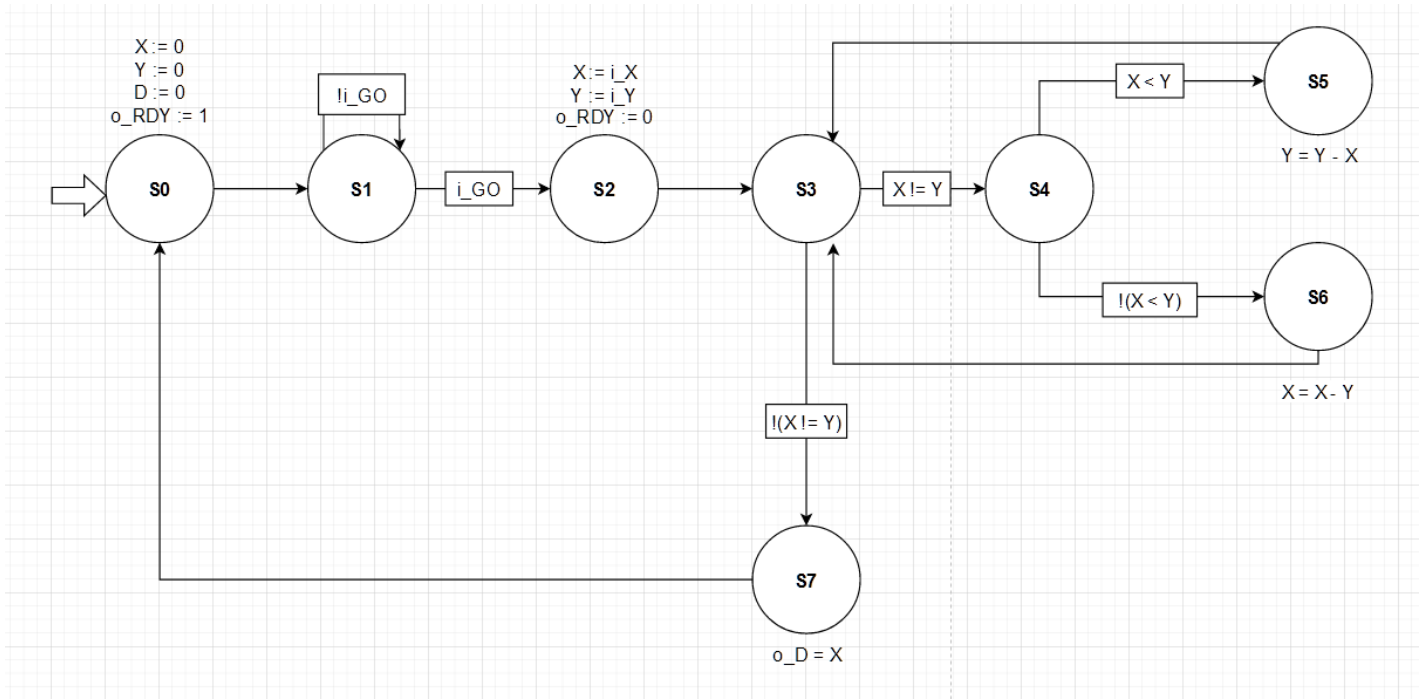
A figura abaixo mostra a correspondência entre os estados da HLSM e das linhas do código fonte:

Figura 2 – Correspondência entre o código em C e a HLSM.

Estado	Descrição	Linhas
S0	Inicia X e Y para 0 e RDY para 1.	1-4
S1	Espera até que i_GO seja 1.	6
S2	Atribui i_X para X, i_Y para Y e 0 para RDY.	8-11
S3	Íncio do loop de cálculo. Realiza “X = Y”.	13
S4	Se X != Y, realiza “X < Y”.	14
S5	Se X != Y e X < Y, então Y recebe Y – X. Volta para o início do loop de cálculo.	15
S6	Se X != Y mas X < Y não é verdadeiro, então X recebe X – Y. Volta para o início do loop de cálculo.	17
S7	Se X = Y, termina o loop de cálculo e atribui X para D.	20

Fonte: O autor.

Figura 3 – HLSM do código em C.



Fonte: O autor.

2.2.3 ETAPA 3 - CRIAR O CAMINHO DE DADOS

Esta etapa consiste na criação de um caminho de dados que irá realizar os processamentos presentes na HLSM. Será necessário o uso de:

- Registradores de 8 bits: Armazenar os valores de X, Y e D;
- Comparadores de magnitude: Comparar X e Y;
- Subtratores: Realizar “ $Y = Y - X$ ” e “ $X = X - Y$ ”;
- Multiplexadores: Para selecionar qual valor será armazenado nos registradores X e Y.

Estes componentes de caminhos de dados já foram desenvolvidos em trabalhos passados, com a exceção do comparador de magnitude de 8 bits.

2.2.3.1 COMPARADOR DE MAGNITUDE DE 1 BIT

O comparador de magnitude 1 bit possui duas entradas de dados (i_A e i_B), três entradas de 1 bit que indicam o resultado da comparação do bit passado (i_GT , i_EQ e i_LT), e possui 3 saídas que indicam o resultado da comparação atual (o_GT , o_EQ e o_LT).

Ele possui essas entradas para se encadear com outros comparadores de 1 bit com o objetivo de formar comparadores de quantidade de bits variáveis de maneira conveniente.

Figura 4 – Entidade e arquitetura do comparador de 1 bit (comparator_1bit.vhd).

```

library IEEE;
use IEEE.std_logic_1164.all;

entity comparator_1bit is
    port
    (
        i_A: in std_logic;
        i_B: in std_logic;
        i_GT: in std_logic;
        i_EQ: in std_logic;
        i_LT: in std_logic;

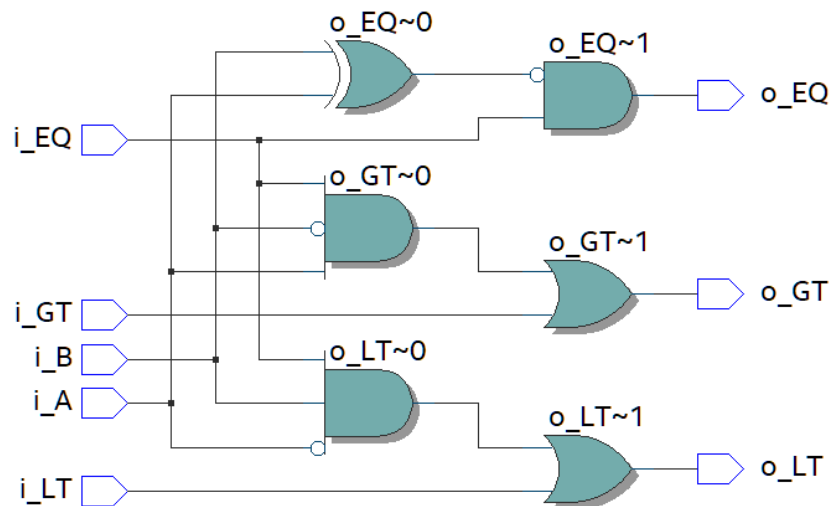
        o_GT: out std_logic;
        o_EQ: out std_logic;
        o_LT: out std_logic
    );
end comparator_1bit;

architecture arch_comparator_1bit of comparator_1bit is

begin
    o_GT <= i_GT or (i_EQ and i_A and not i_B);
    o_EQ <= i_EQ and (i_A xnor i_B);
    o_LT <= i_LT or (i_EQ and (not i_A) and i_B);
end arch_comparator_1bit;

```

Fonte: O autor.

Figura 5 – Diagrama RTL do comparador de 1 bit.

Fonte: O autor.

2.2.3.2 COMPARADOR DE MAGNITUDE DE 8 BITS

O comparador de magnitude 8 bits é composto de 8 comparadores de 1 bit. Internamente, os oito comparadores são encadeados.

Figura 6 – Entidade e arquitetura do comparador de magnitude de 8 bits (comparator_8bit.vhd).

```
library IEEE;
use IEEE.std_logic_1164.all;

entity comparator_8bit is
    port
    (
        i_A: in std_logic_vector (7 downto 0);
        i_B: in std_logic_vector (7 downto 0);
        o_GT: out std_logic;
        o_EQ: out std_logic;
        o_LT: out std_logic
    );
end comparator_8bit;

architecture arch_comparator_8bit of comparator_8bit is

    component comparator_1bit is
        port
        (
            i_A: in std_logic;
            i_B: in std_logic;
            i_GT: in std_logic;
            i_EQ: in std_logic;
            i_LT: in std_logic;
            o_GT: out std_logic;
            o_EQ: out std_logic;
            o_LT: out std_logic
        );
    end component;

    signal w_COMPARATOR_0_GT: std_logic;
    signal w_COMPARATOR_0_EQ: std_logic;
    signal w_COMPARATOR_0_LT: std_logic;

    signal w_COMPARATOR_1_GT: std_logic;
    signal w_COMPARATOR_1_EQ: std_logic;
    signal w_COMPARATOR_1_LT: std_logic;

    signal w_COMPARATOR_2_GT: std_logic;
    signal w_COMPARATOR_2_EQ: std_logic;
    signal w_COMPARATOR_2_LT: std_logic;
```



```

signal w_COMPARATOR_3_GT: std_logic;
signal w_COMPARATOR_3_EQ: std_logic;
signal w_COMPARATOR_3_LT: std_logic;

signal w_COMPARATOR_4_GT: std_logic;
signal w_COMPARATOR_4_EQ: std_logic;
signal w_COMPARATOR_4_LT: std_logic;

signal w_COMPARATOR_5_GT: std_logic;
signal w_COMPARATOR_5_EQ: std_logic;
signal w_COMPARATOR_5_LT: std_logic;

signal w_COMPARATOR_6_GT: std_logic;
signal w_COMPARATOR_6_EQ: std_logic;
signal w_COMPARATOR_6_LT: std_logic;

begin
    u_COMPARATOR_0: comparator_1bit port map(i_A(7), i_B(7), '0', '1', '0',
w_COMPARATOR_0_GT, w_COMPARATOR_0_EQ, w_COMPARATOR_0_LT);
    u_COMPARATOR_1: comparator_1bit port map(i_A(6), i_B(6), w_COMPARATOR_0_GT,
w_COMPARATOR_0_EQ, w_COMPARATOR_0_LT, w_COMPARATOR_1_GT, w_COMPARATOR_1_EQ,
w_COMPARATOR_1_LT);
    u_COMPARATOR_2: comparator_1bit port map(i_A(5), i_B(5), w_COMPARATOR_1_GT,
w_COMPARATOR_1_EQ, w_COMPARATOR_1_LT, w_COMPARATOR_2_GT, w_COMPARATOR_2_EQ,
w_COMPARATOR_2_LT);
    u_COMPARATOR_3: comparator_1bit port map(i_A(4), i_B(4), w_COMPARATOR_2_GT,
w_COMPARATOR_2_EQ, w_COMPARATOR_2_LT, w_COMPARATOR_3_GT, w_COMPARATOR_3_EQ,
w_COMPARATOR_3_LT);
    u_COMPARATOR_4: comparator_1bit port map(i_A(3), i_B(3), w_COMPARATOR_3_GT,
w_COMPARATOR_3_EQ, w_COMPARATOR_3_LT, w_COMPARATOR_4_GT, w_COMPARATOR_4_EQ,
w_COMPARATOR_4_LT);
    u_COMPARATOR_5: comparator_1bit port map(i_A(2), i_B(2), w_COMPARATOR_4_GT,
w_COMPARATOR_4_EQ, w_COMPARATOR_4_LT, w_COMPARATOR_5_GT, w_COMPARATOR_5_EQ,
w_COMPARATOR_5_LT);
    u_COMPARATOR_6: comparator_1bit port map(i_A(1), i_B(1), w_COMPARATOR_5_GT,
w_COMPARATOR_5_EQ, w_COMPARATOR_5_LT, w_COMPARATOR_6_GT, w_COMPARATOR_6_EQ,
w_COMPARATOR_6_LT);
    u_COMPARATOR_7: comparator_1bit port map(i_A(0), i_B(0), w_COMPARATOR_6_GT,
w_COMPARATOR_6_EQ, w_COMPARATOR_6_LT, o_GT, o_EQ, o_LT);
end arch_comparator_8bit;

```

Fonte: O autor.

Figura 7 – Testbench do comparador de magnitude de 8 bits (comparator_8bit_tb.vhd).

```

library IEEE;
use IEEE.std_logic_1164.all;

entity comparator_8bit_tb is
end comparator_8bit_tb;

architecture arch_comparator_8bit_tb of comparator_8bit_tb is

component comparator_8bit is

```

```

port
(
    i_A: in std_logic_vector (7 downto 0);
    i_B: in std_logic_vector (7 downto 0);
    o_GT: out std_logic;
    o_EQ: out std_logic;
    o_LT: out std_logic
);
end component;

signal w_A: std_logic_vector(7 downto 0);
signal w_B: std_logic_vector(7 downto 0);
signal w_GT: std_logic;
signal w_EQ: std_logic;
signal w_LT: std_logic;

begin
    u_COMPARATOR: comparator_8bit port map(w_A, w_B, w_GT, w_EQ, w_LT);

    process
    begin
        -- Test 1 - A = B
        w_A <= "00001000";
        w_B <= "00001000";

        wait for 1 ns;
        assert (w_GT = '0') report "Fail comparator_8bit -> Test 1" severity error;
        assert (w_EQ = '1') report "Fail comparator_8bit -> Test 1" severity error;
        assert (w_LT = '0') report "Fail comparator_8bit -> Test 1" severity error;

        -- Test 2 - A > B
        w_A <= "01011000";
        w_B <= "00101000";

        wait for 1 ns;
        assert (w_GT = '1') report "Fail comparator_8bit -> Test 2" severity error;
        assert (w_EQ = '0') report "Fail comparator_8bit -> Test 2" severity error;
        assert (w_LT = '0') report "Fail comparator_8bit -> Test 2" severity error;

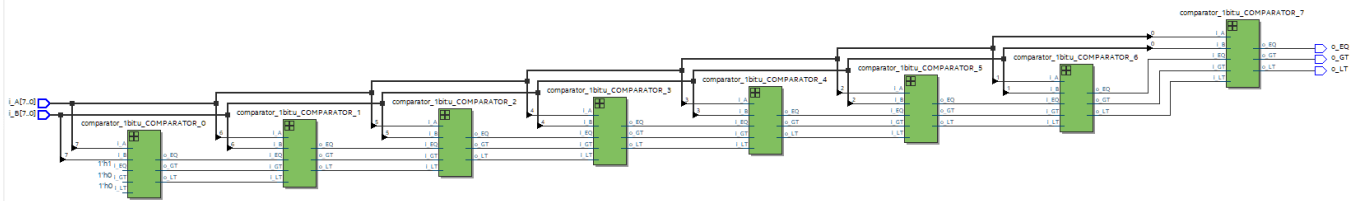
        -- Test 3 - A < B
        w_A <= "00000111";
        w_B <= "00001000";

        wait for 1 ns;
        assert (w_GT = '0') report "Fail comparator_8bit -> Test 3" severity error;
        assert (w_EQ = '0') report "Fail comparator_8bit -> Test 3" severity error;
        assert (w_LT = '1') report "Fail comparator_8bit -> Test 3" severity error;

        wait;
    end process;
end arch_comparator_8bit_tb;

```

Fonte: O autor.

Figura 8 – Diagrama RTL do comparador de magnitude de 8 bits.

Fonte: O autor.

2.2.3.3 CAMINHO DE DADOS

O caminho de dados construído possui 11 entradas e 3 saídas, sendo elas:

Figura 9 – Entradas do caminho de dados.

Entrada	Descrição	Origem
i_CLK	Relógio.	Fora do processador.
i_X	Valor de X entrado pelo usuário.	Fora do processador.
i_X_LOAD	Realiza a carga no registrador X.	Controlador.
i_X_CLRn	Limpa o registrador X quando igual à 0.	Controlador.
i_X_LOAD_MODE	Quando 0, seleciona i_X. Quando 1, seleciona X – Y.	Controlador.
i_Y	Valor de Y entrado pelo usuário.	Fora do processador.
i_Y_LOAD	Realiza a carga no registrador Y.	Controlador.
i_Y_CLRn	Limpa o registrador Y quando igual à 0.	Controlador.
i_Y_LOAD_MODE	Quando 0, seleciona i_Y. Quando 1, seleciona Y – X.	Controlador.
i_D_LOAD	Realiza a carga no registrador D.	Controlador.
i_D_CLRn	Limpa o registrador D quando igual à 0.	Controlador.

Fonte: O autor.

Figura 10 – Saídas do caminho de dados.

Saída	Descrição	Destino
o_X_EQ_Y	1 quando X é igual à Y. 0 quando X não é igual à Y.	Controlador.
o_X_LT_Y	1 quando X é menor que Y. 0 quando X não é menor que Y.	Controlador.
o_D	Valor do resultado do MDC.	Fora do processador.

Fonte: O autor.

No caminho de dados, foram usados dois subtratores, um comparador, dois multiplexadores e três registradores:

- O primeiro subtrator realiza “ $X - Y$ ” e salva o resultado em um sinal chamado “w_X_MINUS_Y”;
- O segundo subtrator realiza “ $Y - X$ ” e salva o resultado em um sinal chamado “w_Y_MINUS_X”;
- O comparador compara X e Y e salva o resultado de “ $X = Y$ ” no sinal “o_X_EQ_Y” e o resultado de “ $X < Y$ ” no sinal “o_X_LT_Y”. O resultado de “ $X > Y$ ” é descartado.
- O primeiro multiplexador seleciona entre

Figura 11 – Entidade e arquitetura do caminho de dados (mdc_datapath.vhd).

```
library IEEE;
use IEEE.std_logic_1164.all;

entity mdc_datapath is
  port
  (
    i_CLK: in std_logic;

    -- Data inputs
    i_X: in std_logic_vector (7 downto 0);
    i_X_LOAD: in std_logic;
    i_X_CLRn: in std_logic;
    i_X_LOAD_MODE: in std_logic;

    i_Y: in std_logic_vector (7 downto 0);
    i_Y_LOAD: in std_logic;
    i_Y_CLRn: in std_logic;
    i_Y_LOAD_MODE: in std_logic;

    i_D_LOAD: in std_logic;
    i_D_CLRn: in std_logic;

    -- Data outputs
    o_X_EQ_Y: out std_logic;
    o_X_LT_Y: out std_logic;
    o_D: out std_logic_vector (7 downto 0)
  );
end mdc_datapath;

architecture arch_mdc_datapath of mdc_datapath is
  component register_8bit is
    port
    (
      i_A: in std_logic_vector (7 downto 0);
      i_LOAD: in std_logic;
      i_CLRn: in std_logic;
      i_CLK: in std_logic;
      o_R: out std_logic_vector (7 downto 0)
    );
```

```

end component;

component mux_2x8bit is
  port
  (
    i_A: in std_logic_vector (7 downto 0);
    i_B: in std_logic_vector (7 downto 0);
    i_SEL: in std_logic;
    o_R: out std_logic_vector (7 downto 0)
  );
end component;

component subtractor_8bit is
  port
  (
    i_A: in std_logic_vector (7 downto 0);
    i_B: in std_logic_vector (7 downto 0);
    o_R: out std_logic_vector (7 downto 0);
    o_CARRY_OUT: out std_logic
  );
end component;

component comparator_8bit is
  port
  (
    i_A: in std_logic_vector (7 downto 0);
    i_B: in std_logic_vector (7 downto 0);
    o_GT: out std_logic;
    o_EQ: out std_logic;
    o_LT: out std_logic
  );
end component;

signal w_MUX_X_OUT: std_logic_vector (7 downto 0);
signal w_X_MINUS_Y: std_logic_vector (7 downto 0);

signal w_MUX_Y_OUT: std_logic_vector (7 downto 0);
signal w_Y_MINUS_X: std_logic_vector (7 downto 0);

signal w_X: std_logic_vector (7 downto 0);
signal w_Y: std_logic_vector (7 downto 0);

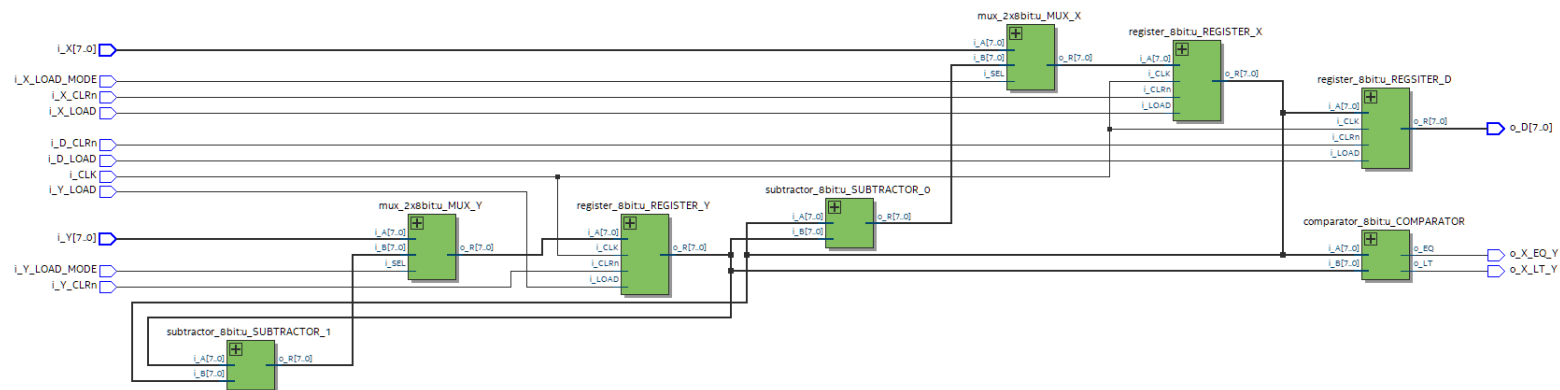
begin
  u_SUBTRACTOR_0: subtractor_8bit port map (w_X, w_Y, w_X_MINUS_Y, open);
  u_SUBTRACTOR_1: subtractor_8bit port map (w_Y, w_X, w_Y_MINUS_X, open);
  u_COMPARATOR: comparator_8bit port map (w_X, w_Y, open, o_X_EQ_Y, o_X_LT_Y);

  u_MUX_X: mux_2x8bit port map (i_X, w_X_MINUS_Y, i_X_LOAD_MODE, w_MUX_X_OUT);
  u_MUX_Y: mux_2x8bit port map (i_Y, w_Y_MINUS_X, i_Y_LOAD_MODE, w_MUX_Y_OUT);

  u_REGISTER_X: register_8bit port map (w_MUX_X_OUT, i_X_LOAD, i_X_CLRn, i_CLK, w_X);
  u_REGISTER_Y: register_8bit port map (w_MUX_Y_OUT, i_Y_LOAD, i_Y_CLRn, i_CLK, w_Y);
  u_REGSITER_D: register_8bit port map (w_X, i_D_LOAD, i_D_CLRn, i_CLK, o_D);
end arch_mdc_datapath;

```

Fonte: O autor.

Figura 12 – Diagrama RTL do caminho de dados.

Fonte: O autor.

2.2.4. ETAPA 4 - DERIVAR A FSM DO CONTROLADOR

Esta etapa consiste na tradução da HLSM do controlador para uma FSM. Para isso, as atribuições e operações devem ser trocadas por atribuições aos sinais que limpam ou carregam os registradores correspondentes.

Alguns estados adicionais foram adicionados para evitar a leitura de valores desatualizados. O valor dos sinais de limpeza (CLRn) por padrão são iguais à 1 quando não mencionados no estado.

Figura 13 – Entradas do controlador.

Entrada	Descrição	Origem
i_CLK	Relógio.	Fora do processador.
i_GO	Sinal para começar o processamento.	Fora do processador.
i_X_EQ_Y	Resultado de "X = Y".	Caminho de dados.
i_X_LT_Y	Resultado de "X < Y".	Caminho de dados.

Fonte: O autor.

Figura 14 – Saídas do controlador.

Saída	Descrição	Destino
o_RDY	1 quando o processador está esperando para iniciar. 0 durante o processamento.	Fora do processador.
o_X_CLRn	Limpa o registrador X quando igual à 0.	Caminho de dados.
o_X_LOAD	Realiza a carga no registrador X.	Caminho de dados
o_X_LOAD_MODE	0 simboliza a carga do valor de i_X. 1 simboliza a carga do valor de X - Y.	Caminho de dados
o_Y_CLRn	Limpa o registrador Y quando igual à 0.	Caminho de dados.
o_Y_LOAD	Realiza a carga no registrador Y.	Caminho de dados
o_Y_LOAD_MODE	0 simboliza a carga do valor de i_Y.	Caminho de dados

	1 simboliza a carga do valor de $Y - X$.	
o_D_CLRn	Limpa o registrador D quando igual à 0.	Caminho de dados.
o_D_LOAD	Realiza a carga no registrador D.	Caminho de dados

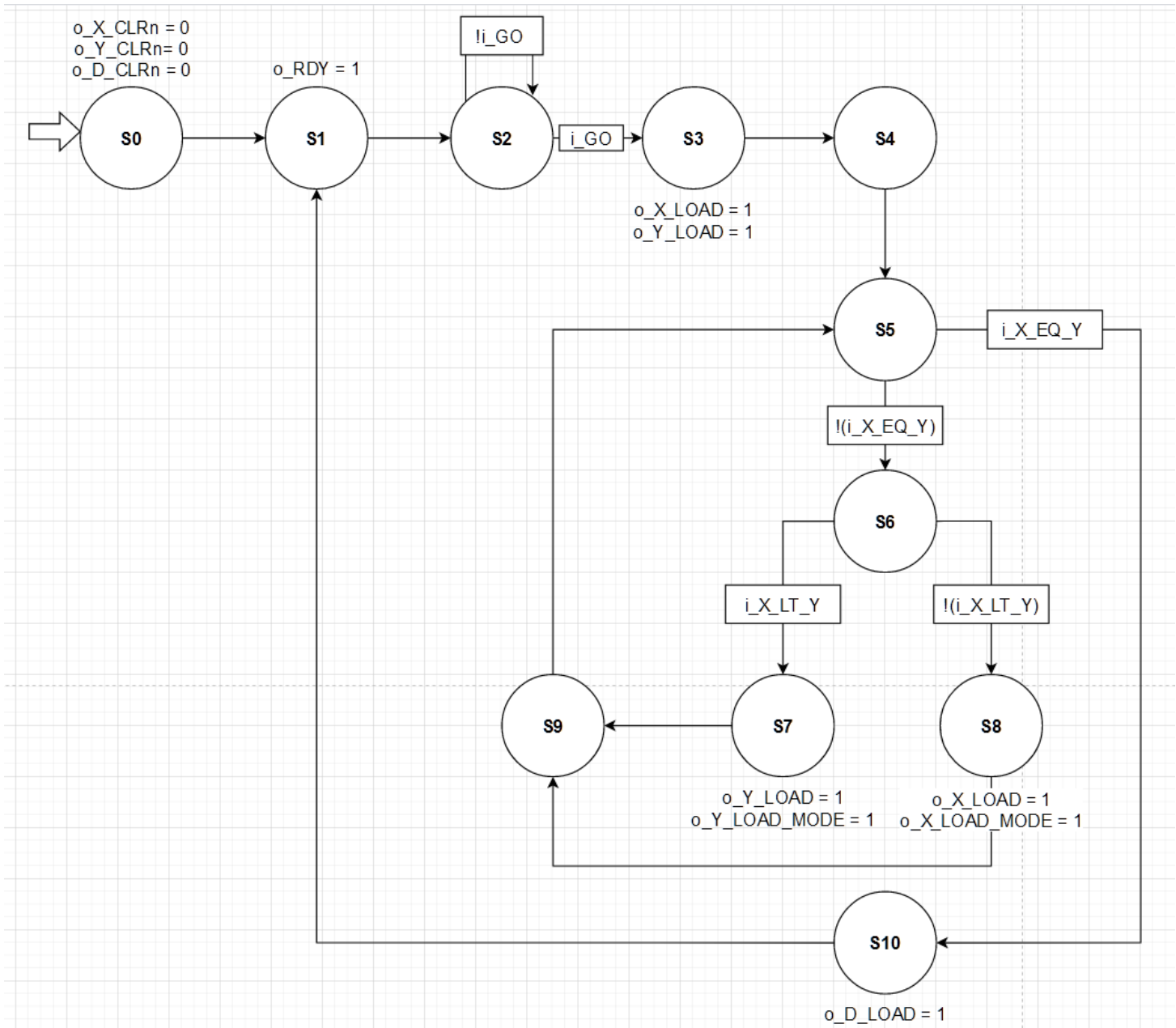
Fonte: O autor.

Figura 15 – Correspondência entre o código em C e a FSM.

Estado	Descrição	Linhas
S0	Inicia X, Y e D para 0.	1
S1	Inicia o_RDY para 1.	3
S2	Espera até que i_GO seja 1.	4
S3	Aciona o_X_LOAD e o_Y_LOAD para atribuir i_X para X e i_Y para Y.	5-6
S4	Espera para atualizar os valores de X e Y.	-
S5	Checa se o sinal i_X_EQ_Y recebido é verdadeiro.	8
S6	Checa se o sinal i_X_LT_Y recebido é verdadeiro.	9
S7	Se sim, carrega Y com o valor de $Y - X$, pois o_Y_LOAD_MODE é 1.	10
S8	Se não, carrega X com o valor de $X - Y$, pois o_X_LOAD_MODE é 1.	12
S9	Espera para atualizar os valores de X e Y. Reinicia o loop de cálculo.	-
S10	Aciona o_D_LOAD para atribuir X a D. Reinicia o loop principal.	14

Fonte: O autor.

Figura 16 – FSM do controlador.



Fonte: O autor.

Figura 17 – Entidade e arquitetura do controlador (mdc_control.vhd).

```

library IEEE;
use IEEE.std_logic_1164.all;

entity mdc_control is
  port
  (
    -- Inputs
    i_CLK: in std_logic;
    i_GO: in std_logic;
    i_X_EQ_Y: in std_logic;
    i_X_LT_Y: in std_logic;
  
```



```

        -- Outputs
        o_RDY: out std_logic;

        o_X_CLRn: out std_logic;
        o_X_LOAD: out std_logic;
        o_X_LOAD_MODE: out std_logic;

        o_Y_CLRn: out std_logic;
        o_Y_LOAD: out std_logic;
        o_Y_LOAD_MODE: out std_logic;

        o_D_CLRn: out std_logic;
        o_D_LOAD: out std_logic
    );
end mdc_control;

architecture arch_mdc_control of mdc_control is

type t_STATE is (s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10);
signal r_STATE: t_STATE;
signal w_NEXT_STATE: t_STATE;

begin
    -- State register
    p_CHANGE_STATE: process(i_CLK) begin
        if (rising_edge(i_CLK)) then
            r_STATE <= w_NEXT_STATE;
        end if;
    end process;

    -- Combinational logic
    p_NEXT_STATE: process(r_STATE, i_GO, i_X_EQ_Y, i_X_LT_Y) begin
        case (r_STATE) is
            when s0 => w_NEXT_STATE <= s1;
            when s1 => w_NEXT_STATE <= s2;
            when s2 => if (i_GO = '1') then w_NEXT_STATE <= s3; else w_NEXT_STATE
<= s2; end if;
            when s3 => w_NEXT_STATE <= s4;
            when s4 => w_NEXT_STATE <= s5;
            when s5 => if (i_X_EQ_Y = '0') then w_NEXT_STATE <= s6; else
w_NEXT_STATE <= s10; end if;
            when s6 => if (i_X_LT_Y = '1') then w_NEXT_STATE <= s7; else
w_NEXT_STATE <= s8; end if;
            when s7 => w_NEXT_STATE <= s9;
            when s8 => w_NEXT_STATE <= s9;
            when s9 => w_NEXT_STATE <= s5;
            when s10 => w_NEXT_STATE <= s1;
            when others => w_NEXT_STATE <= s0;
        end case;
    end process;

    p_OUTPUT: process(r_STATE) begin
        o_RDY <= '0';

        o_X_CLRn <= '1';
        o_X_LOAD <= '0';
        o_X_LOAD_MODE <= '0';
    end process;
end arch_mdc_control;

```

```

o_Y_CLRn <= '1';
o_Y_LOAD <= '0';
o_Y_LOAD_MODE <= '0';

o_D_CLRn <= '1';
o_D_LOAD <= '0';

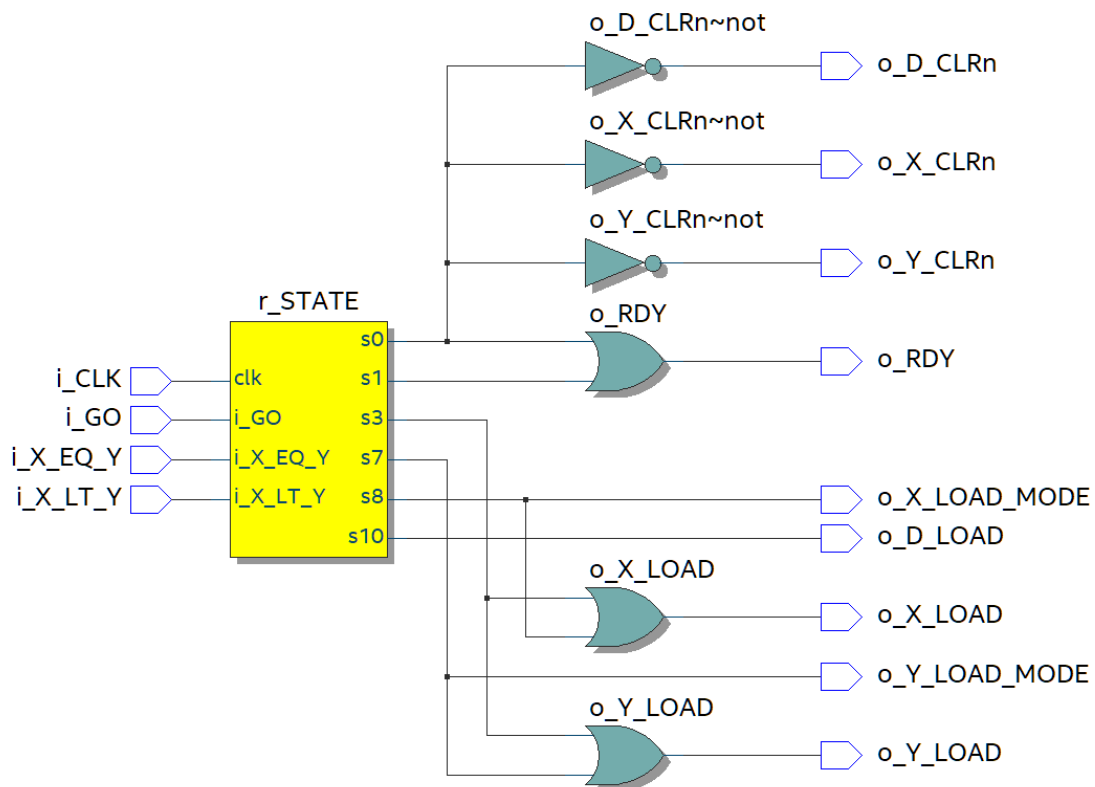
case (r_STATE) is
  when s0 => o_X_CLRn <= '0'; o_Y_CLRn <= '0'; o_D_CLRn <= '0'; o_RDY
<= '1';

  when s1 => o_RDY <= '1';
  when s2 => NULL;
  when s3 => o_X_LOAD <= '1'; o_Y_LOAD <= '1';
  when s4 => NULL;
  when s5 => NULL;
  when s6 => NULL;
  when s7 => o_Y_LOAD <= '1'; o_Y_LOAD_MODE <= '1';
  when s8 => o_X_LOAD <= '1'; o_X_LOAD_MODE <= '1';
  when s9 => NULL;
  when s10 => o_D_LOAD <= '1';
  when others => NULL;
end case;
end process;
end arch_mdc_control;

```

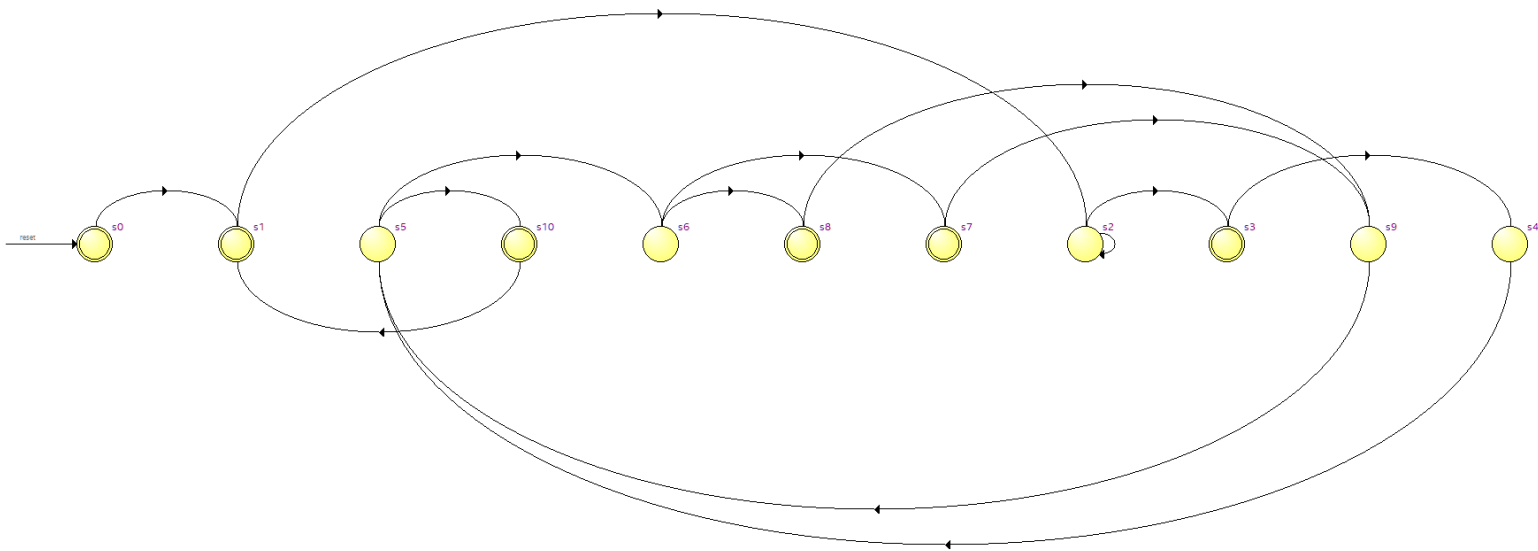
Fonte: O autor.

Figura 18 – Diagrama RTL do controlador.



Fonte: O autor.

Figura 19 – Máquina de estados gerada pelo Intel Quartus Prime.



Fonte: O autor.

2.2.5 ETAPA 5 - CONEXÃO DO CONTROLADOR E CAMINHO DE DADOS

Esta etapa consiste na conexão do controlador e do caminho de dados. Para isso, basta simplesmente conectar os sinais correspondentes.

Figura 20 – Entidade e arquitetura do processador (mdc_top.vhd).

```
library IEEE;
use IEEE.std_logic_1164.all;

entity mdc_top is
  port
  (
    i_X: in std_logic_vector (7 downto 0);
    i_Y: in std_logic_vector (7 downto 0);
    i_GO: in std_logic;
    i_CLK: in std_logic;

    o_D: out std_logic_vector (7 downto 0);
    o_RDY: out std_logic
  );
end mdc_top;

architecture arch_mdc_top of mdc_top is
  component mdc_control is
    port
    (
      -- Inputs
      i_CLK: in std_logic;
      i_GO: in std_logic;
      i_X_EQ_Y: in std_logic;
      i_X_LT_Y: in std_logic;
```

```

        -- Outputs
        o_RDY: out std_logic;

        o_X_CLRn: out std_logic;
        o_X_LOAD: out std_logic;
        o_X_LOAD_MODE: out std_logic;

        o_Y_CLRn: out std_logic;
        o_Y_LOAD: out std_logic;
        o_Y_LOAD_MODE: out std_logic;

        o_D_CLRn: out std_logic;
        o_D_LOAD: out std_logic;
    );
end component;

component mdc_datapath is
    port
    (
        i_CLK: in std_logic;

        -- Data inputs
        i_X: in std_logic_vector (7 downto 0);
        i_X_LOAD: in std_logic;
        i_X_CLRn: in std_logic;
        i_X_LOAD_MODE: in std_logic;

        i_Y: in std_logic_vector (7 downto 0);
        i_Y_LOAD: in std_logic;
        i_Y_CLRn: in std_logic;
        i_Y_LOAD_MODE: in std_logic;

        i_D_LOAD: in std_logic;
        i_D_CLRn: in std_logic;

        -- Data outputs
        o_X_EQ_Y: out std_logic;
        o_X_LT_Y: out std_logic;
        o_D: out std_logic_vector (7 downto 0)
    );
end component;

-- Clock signal
signal w_CLK: std_logic;

-- Controller outputs to Datapath
signal w_X_CLRn: std_logic;
signal w_X_LOAD: std_logic;
signal w_X_LOAD_MODE: std_logic;

signal w_Y_CLRn: std_logic;
signal w_Y_LOAD: std_logic;
signal w_Y_LOAD_MODE: std_logic;

signal w_D_CLRn: std_logic;
signal w_D_LOAD: std_logic;

```

```

-- Datapath outputs to Controller
signal w_X_EQ_Y: std_logic;
signal w_X_LT_Y: std_logic;

begin
    u_MDC_CONTROL: mdc_control port map (
        i_CLK,
        i_GO,
        w_X_EQ_Y,
        w_X_LT_Y,

        o_RDY,

        w_X_CLRn,
        w_X_LOAD,
        w_X_LOAD_MODE,

        w_Y_CLRn,
        w_Y_LOAD,
        w_Y_LOAD_MODE,

        w_D_CLRn,
        w_D_LOAD
    );

    u_MDC_DATAPATH: mdc_datapath port map (
        i_CLK,

        i_X,
        w_X_LOAD,
        w_X_CLRn,
        w_X_LOAD_MODE,

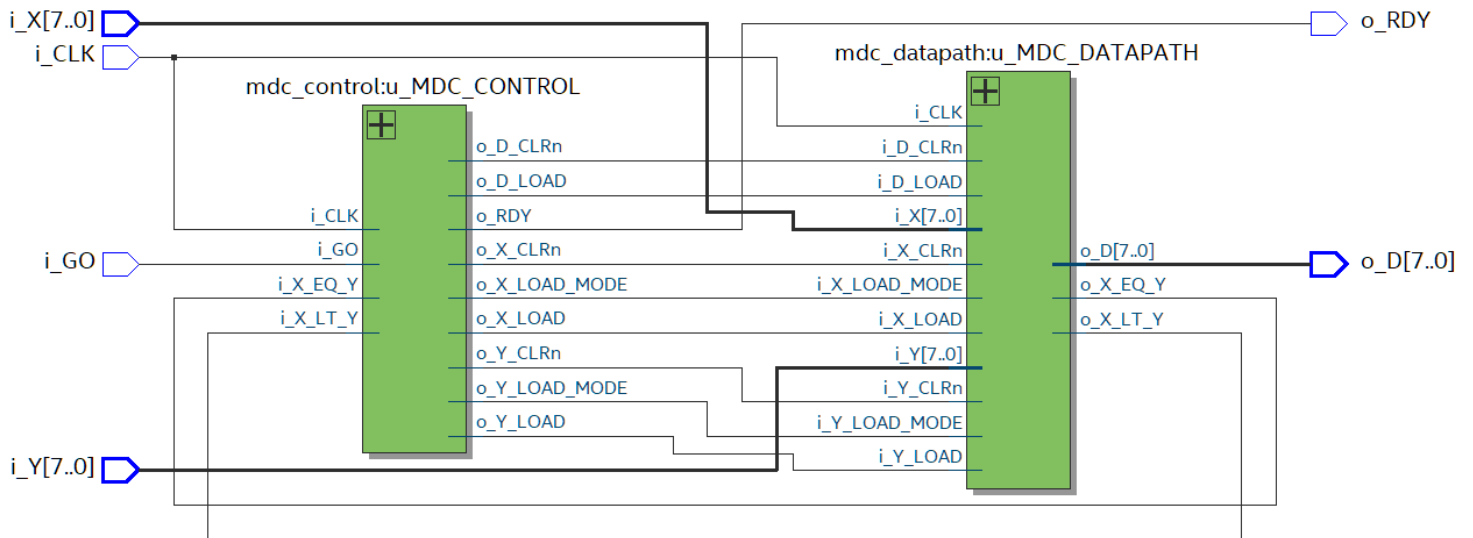
        i_Y,
        w_Y_LOAD,
        w_Y_CLRn,
        w_Y_LOAD_MODE,

        w_D_LOAD,
        w_D_CLRn,

        w_X_EQ_Y,
        w_X_LT_Y,
        o_D
    );
end arch_mdc_top;

```

Fonte: O autor.

Figura 21 – Diagrama RTL do processador.

Fonte: O autor.

2.3 ANÁLISE E TESTES

Foram realizados três testes de cálculo de MDC no processador. Cada teste consiste nas seguintes etapas:

- Entrada de dois valores de dados;
- Acionamento do sinal de começo (`i_GO`);
- Espera de 80ns, para que o resultado seja computado;
- Verificação do resultado.

Figura 22 – Testes do processador.

X	Y	D (resultado esperado)
12	4	4
16	40	8
5	5	5

Fonte: O autor.

Figura 23 – Testbench do processador (`mdc_top_tb.vhd`).

```

library IEEE;
use IEEE.std_logic_1164.all;

entity mdc_top_tb is
end mdc_top_tb;

architecture arch_mdc_top_tb of mdc_top_tb is

component mdc_top is
  port
  (
    i_X: in std_logic_vector (7 downto 0);
    i_Y: in std_logic_vector (7 downto 0);
  
```

```

        i_GO: in std_logic;
        i_CLK: in std_logic;

        o_D: out std_logic_vector (7 downto 0);
        o_RDY: out std_logic
    );
end component;

signal w_X: std_logic_vector (7 downto 0);
signal w_Y: std_logic_vector (7 downto 0);
signal w_GO: std_logic;
signal w_CLK: std_logic;
signal w_D: std_logic_vector (7 downto 0);
signal w_RDY: std_logic;

begin
    u_MDC_TOP: mdc_top port map (w_X, w_Y, w_GO, w_CLK, w_D, w_RDY);

    process
    begin
        -- Test 1 (12 and 4)
        w_X <= "00001100";
        w_Y <= "00000100";

        w_GO <= '1';
        wait for 8 ns;
        w_GO <= '0';

        wait for 80 ns;
        assert (w_D = "00000100") report "Fail mdc_top_tb -> Test 1" severity error;

        -- Test 2 (16 and 40)
        w_X <= "00010000";
        w_Y <= "00101000";

        w_GO <= '1';
        wait for 8 ns;
        w_GO <= '0';

        wait for 80 ns;
        assert (w_D = "00001000") report "Fail mdc_top_tb -> Test 2" severity error;

        -- Test 3 (5 and 5)
        w_X <= "00000101";
        w_Y <= "00000101";

        w_GO <= '1';
        wait for 8 ns;
        w_GO <= '0';

        wait for 80 ns;
        assert (w_D = "00000101") report "Fail mdc_top_tb -> Test 3" severity error;

        wait;
    end process;
end arch_mdc_top_tb;

```

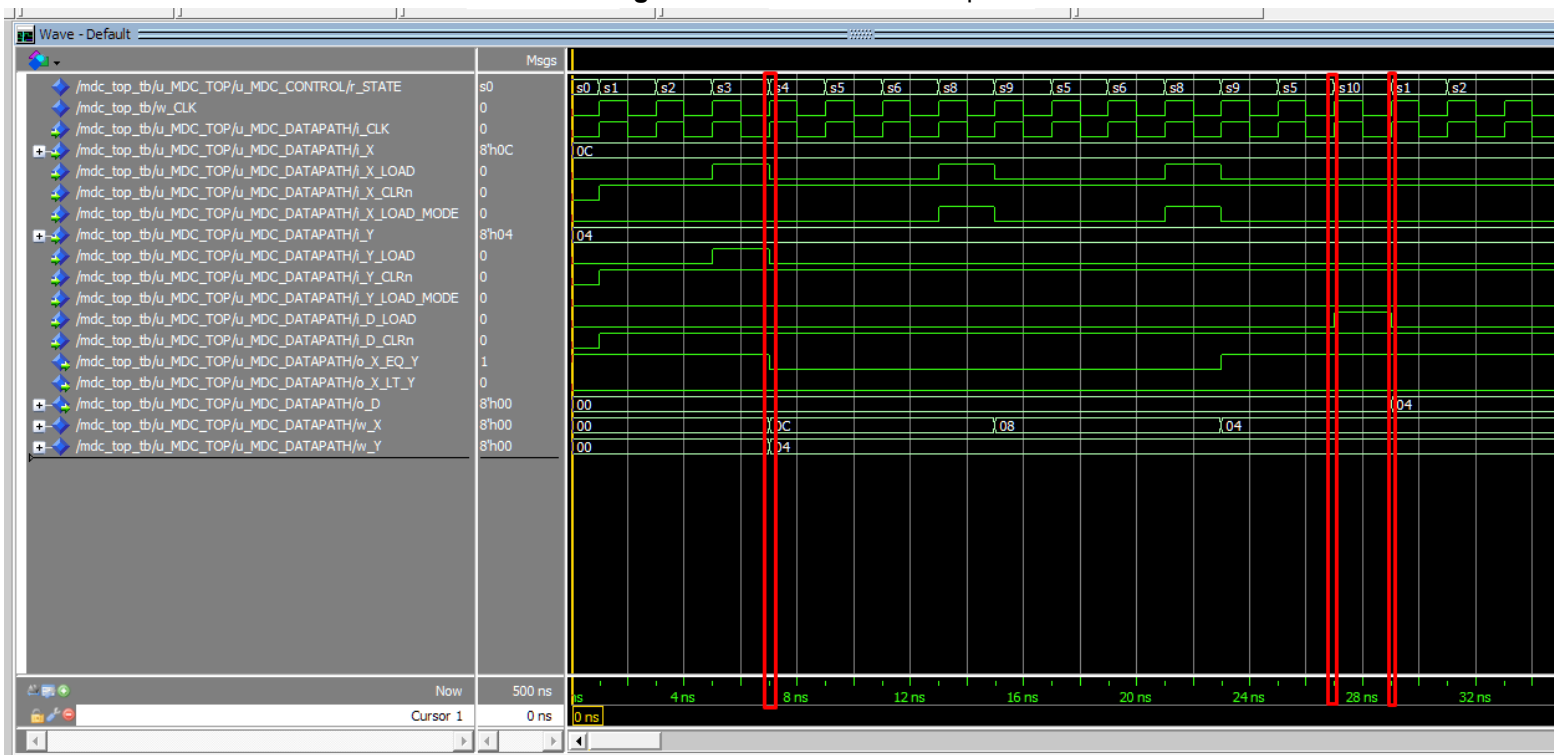
Fonte: O autor.

2.3.1 PRIMEIRO TESTE

O primeiro teste usou os valores 12 para X e 4 para Y, com o resultado esperado igual a 4.

- A máquina começa no estado s0;
- No instante 7ns (transição do estado s3 à s4), os valor de i_X (0x0C) é atribuído para X e o valor de i_Y (0x04) é atribuído para Y;
- O estado atual fica alterando entre s5, s6, s8 e s9 enquanto subtrai os valores de X;
- No instante 27ns se transiciona para o estado s10, acionando o sinal para carregar o registrador de resultado (i_D_LOAD); O registrador D é carregado e o seu valor passa à ser 0x04 no próximo estado (instante 29ns);
- Após, o processador retorna ao estado s1, onde espera novamente pelo sinal i_GO;

Figura 24 – Primeiro teste do processador.



Fonte: O autor.

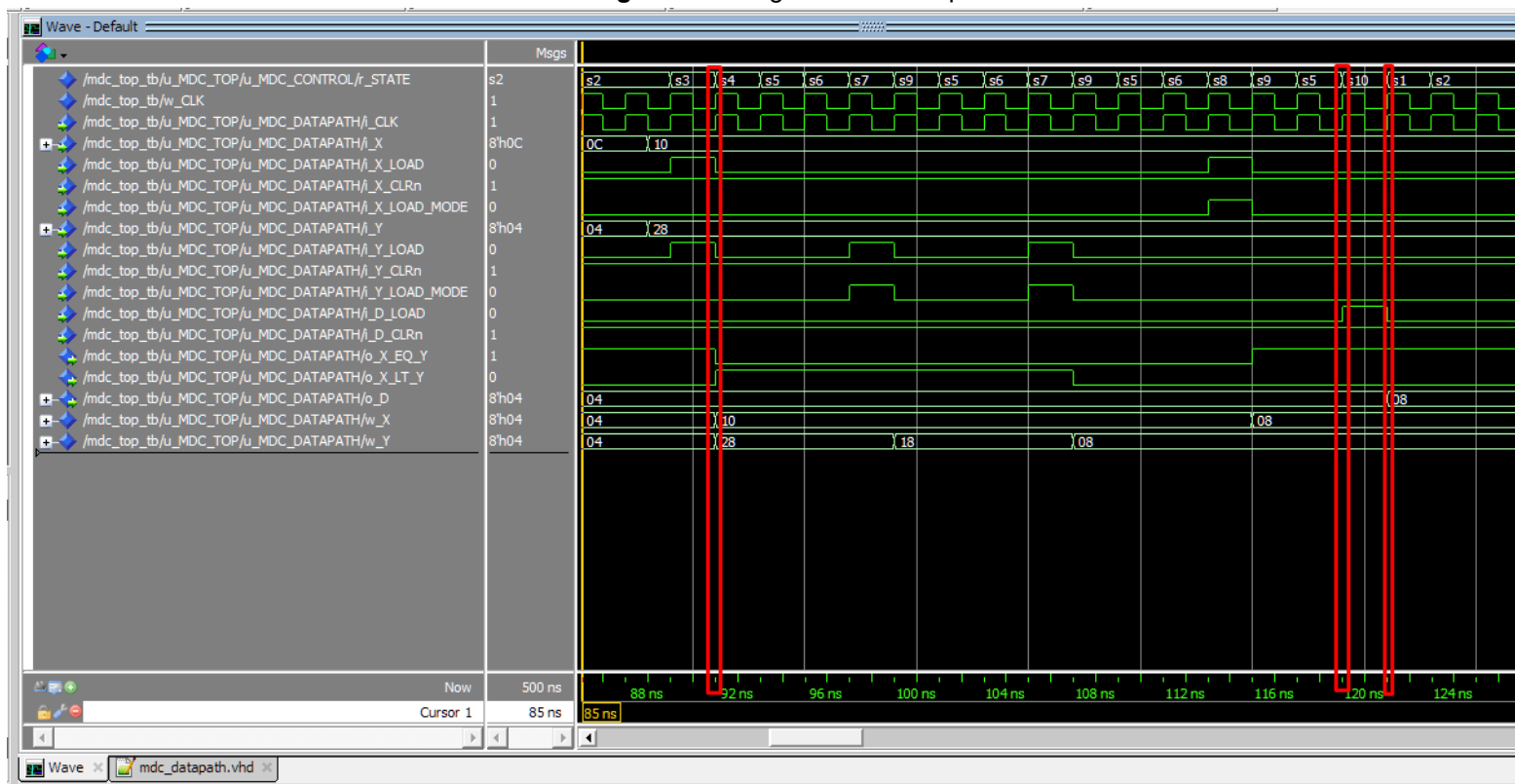
2.3.2 SEGUNDO TESTE

O segundo teste usou os valores 16 para X e 40 para Y, com o resultado esperado igual a 8.

- A máquina começa no estado s0;
- No instante 91ns (transição do estado s3 à s4), os valor de i_X (0x10) é atribuído para X e o valor de i_Y (0x28) é atribuído para Y;

- O estado atual fica alterando entre s5, s6, s7, s8 e s9 enquanto subtrai os valores de X e de Y;
- No instante 119ns se transiciona para o estado s10, acionando o sinal para carregar o registrador de resultado (i_D_LOAD); O registrador D é carregado e o seu valor passa à ser 0x08 no próximo estado (instante 121ns);
- Após, o processador retorna ao estado s1, onde espera novamente pelo sinal i_GO;

Figura 25 – Segundo teste do processador.



Fonte: O autor.

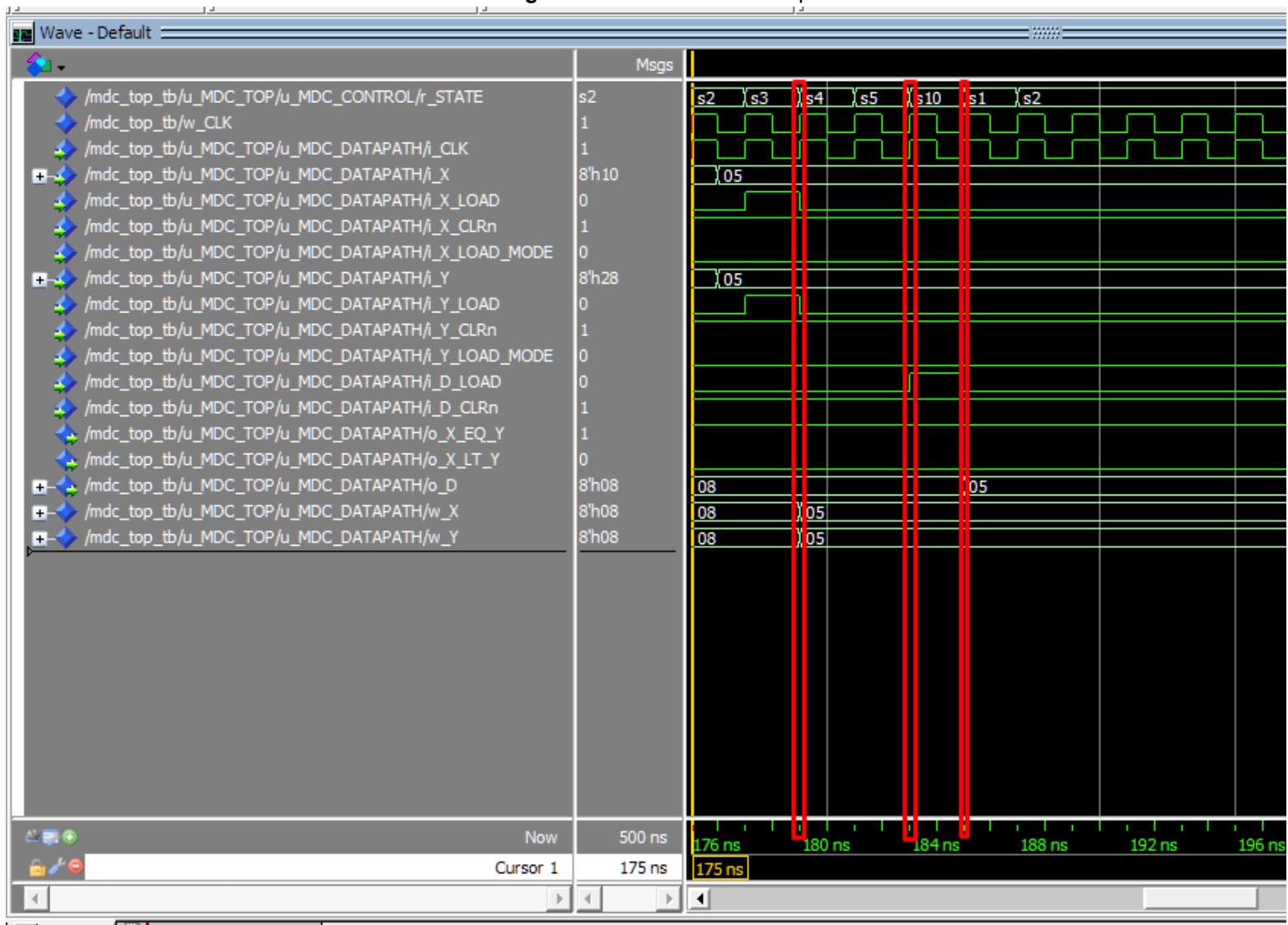
2.3.3 TERCEIRO TESTE

O terceiro teste usou os valores 5 para X e 5 para Y, com o resultado esperado igual a 5.

- A máquina começa no estado s0;
- No instante 179ns (transição do estado s3 à s4), os valor de i_X (0x05) é atribuído para X e o valor de i_Y (0x05) é atribuído para Y;
- O estado atual passa por s5 e então vai diretamente para s10, visto que X e Y são inicialmente iguais;
- No instante 183ns se transiciona para o estado s10, acionando o sinal para carregar o registrador de resultado (i_D_LOAD); O registrador D é carregado e o seu valor passa à ser 0x05 no próximo estado (instante 185ns);

- Após, o processador retorna ao estado s1, onde espera novamente pelo sinal i_GO;

Figura 26 – Terceiro teste do processador.



Fonte: O autor.